

AUTONOMOUS RESEARCH REPORT AGENT

Phase 1 - Team Explanation & Viva Preparation Guide

Phase 1 in one sentence

User gives a research topic -> Tavily finds relevant web information -> Python passes that information to Gemini -> Gemini converts it into a readable, cited summary -> the terminal displays the summary and source URLs.

Current stack

Component	Role
Python	Controls the flow and connects the services.
Tavily API	Searches the web and returns result titles, URLs and extracted content.
Gemini 3.5 Flash-Lite	Reads retrieved evidence and produces a concise research summary.
.env	Stores API keys outside source code.
.venv	Keeps project Python packages isolated.

Prepared from the working Phase 1 implementation and the original major-project plan.

1. What Phase 1 Was Supposed to Achieve

The original project plan defines Phase 1 as the foundation: create a basic end-to-end pipeline, connect the LLM and search services, accept a topic, perform a search, generate a short summary, and handle failed calls. The planned deliverable is a CLI tool that accepts a topic and prints a summary with a source.

What we achieved

The working program accepts a topic in the terminal, uses Tavily to retrieve up to three web results, passes their extracted content to Gemini, asks Gemini to summarize only that evidence, prints inline labels such as [Source 1], and finally prints the real source titles and URLs.

2. The Program Flow - Easy Version

Step	Action
1	You enter a topic Example: Indian cricket team in ICC ODI World Cup 2023
2	Tavily searches the web Returns relevant results containing title, URL and extracted content.
3	Python builds research context Combines Source 1, Source 2 and Source 3 into evidence for the LLM.
4	Gemini reads the evidence The prompt tells it not to invent facts and to cite supplied sources.
5	The terminal shows the result Readable summary + [Source n] references + source URLs.

The easiest way to explain it

Tavily = researcher. It finds information.

Gemini = writer/reasoner. It converts retrieved information into readable text.

Python = manager. It controls the order and passes data between components.

3. What Each Function in main.py Does

Function	Input	Job	Output
load_clients()	.env	Loads keys and creates Gemini/Tavily clients.	API clients
search_web()	Topic	Calls Tavily with basic search and max_results=3.	Search results
build_research_context()	Results	Formats titles, URLs and content as Source 1, 2, 3.	Context text
generate_summary()	Topic + results	Builds prompt and calls Gemini.	Summary
print_sources()	Results	Prints source labels, titles and URLs.	Source list
main()	User input	Controls the sequence and catches errors.	CLI output

4. What Tavily Actually Returns

Tavily gives the Python program structured search results. The important fields used in Phase 1 are title, url and content. The content field contains relevant extracted text from the search result; it is not necessarily the entire webpage.

Conceptual result

```
{
  "title": "ICC Cricket World Cup 2023",
  "url": "https://...",
  "content": "Relevant extracted information..."
}
```

Important viva distinction

Search and generation are separate. Tavily retrieves evidence from the web. Gemini receives that evidence and synthesizes it. Gemini is not using Google Search grounding in the current Phase 1 version.

5. Walkthrough of the Successful Test

Input topic: Indian cricket team in ICC ODI World Cup 2023

In the successful run, Tavily returned three results, including a 2023 Cricket World Cup page and ESPNcricinfo results. Gemini produced a summary about India's group-stage performance, tournament finish and notable player performances, using labels such as [Source 1], [Source 2] and [Source 3].

Stage	What happened
Input	User typed a research topic in the CLI.
Retrieval	Tavily searched and returned up to three relevant results.
Context	Python combined title, URL and content for each result.
Prompting	Gemini was told to use only supplied information, avoid unsupported facts and cite [Source n].
Generation	Gemini converted the evidence into 2-4 readable paragraphs.
Output	The terminal printed the summary and source URLs.

6. What Phase 1 Does NOT Do Yet

- Split a broad topic into sub-questions.
- Research each sub-question independently.
- Verify every claim across multiple sources.
- Assign confidence scores or systematically detect contradictions.
- Produce the final structured Markdown/PDF research report.
- Critique its own report and automatically re-search weak areas.

Why this matters

Phase 1 is intentionally a single-shot foundation. The full project becomes genuinely agentic when planning, verification, synthesis and the Critique -> Search feedback loop are added.

7. Viva Preparation - Core Questions

Q1. What is the objective of Phase 1?

To prove the basic end-to-end research pipeline works: accept a topic, retrieve web information, send it to an LLM and display a readable summary with sources.

Q2. Why are you using Tavily?

Tavily is the web-retrieval component. It searches for the topic and returns structured results containing source titles, URLs and relevant extracted content.

Q3. Why are you using Gemini?

Gemini is the language model. It reads the retrieved evidence, understands it and synthesizes it into a concise human-readable summary.

Q4. Does Gemini search the web in your current program?

No. Tavily performs web retrieval. Gemini receives Tavily's returned content through the prompt.

Q5. What is Python's role?

Python is the controller/orchestrator: it loads configuration, accepts input, calls Tavily, formats context, calls Gemini, handles errors and prints output.

Q6. Why store keys in .env?

To keep secrets separate from source code and reduce the risk of exposing keys in GitHub or shared code.

Q7. Why use .venv?

It isolates this project's dependencies from other Python projects.

Q8. Is Phase 1 already a multi-agent system?

No. It is a fixed single-shot pipeline and foundation. Specialized agents are added later.

8. Viva Preparation - Deeper Questions

Q9. Retrieval vs generation?

Retrieval finds relevant external evidence; generation uses an LLM to understand and express that evidence. Tavily handles retrieval and Gemini handles generation.

Q10. How do citations work now?

The prompt asks Gemini to refer to supplied evidence as [Source 1], [Source 2], etc. Python separately prints matching source titles and URLs. This is basic citation mapping, not full claim-level verification.

Q11. Can Tavily results be wrong?

Yes. Retrieval relevance does not guarantee credibility or factual correctness. A later Verifier Agent will cross-check facts, flag contradictions and assign confidence.

Q12. Why not ask Gemini directly?

Direct answers may be outdated or unsupported. Retrieval first gives the system external evidence that can be displayed and later verified.

Q13. What if an API call fails?

The current pipeline catches exceptions and prints an error. Later versions can add retries and fallback strategies.

Q14. Why only three results?

Phase 1 is a small proof of concept. Three results demonstrate multi-source input while keeping retrieval simple.

Q15. Biggest Phase 1 limitation?

It follows a fixed Search -> Generate -> Print sequence and cannot decide to re-research weak areas.

Q16. What makes the final system agentic?

Specialized agents perform planning, searching, verification, synthesis and critique; the Critique Agent can trigger targeted re-search.

Q17. What is next?

Phase 2: a Planner Agent that decomposes one broad topic into about 3-5 relevant, non-redundant sub-questions.

9. 60-Second Team / Viva Explanation

Suggested explanation

"In Phase 1, we built the foundation of our Autonomous Research Report Agent. The user enters a research topic through the command line. Python sends that topic to Tavily, which searches the web and returns relevant results containing titles, URLs and extracted content. Python formats those results as research context and sends them to Gemini. Gemini is instructed to use only the supplied evidence, avoid unsupported claims and reference the evidence using Source 1, Source 2 and Source 3. Finally, Python prints the generated summary and matching source URLs. At this stage the flow is fixed, so it is not yet the full multi-agent system. In Phase 2 we will add a Planner Agent that breaks the topic into sub-questions, followed later by verification, synthesis and critique agents."

10. Quick Demo Checklist

- Activate .venv.
- Confirm .env contains GEMINI_API_KEY and TAVILY_API_KEY, but never show the actual keys.
- Run: python main.py
- Enter a clear research topic.
- Point out 'Searching the web...' - this is Tavily.
- Point out 'Generating summary with Gemini...' - this is the LLM stage.
- Show [Source 1], [Source 2], etc. in the summary.
- Show the source URLs at the bottom.
- Explain that Phase 1 is fixed and Phase 2 adds planning.

If an examiner asks: 'What did YOU build?'

A strong answer is: "We built the orchestration around the APIs." Tavily and Gemini are external services, but our Python application controls input, retrieval, evidence formatting, LLM constraints, citation mapping, error handling and output. Later phases add our multi-agent control logic on top of this foundation.